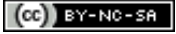


AUTODESK
Instructables

Focus Dial — a Digital Distraction Blocker

By [salimbenbouz](#) in [CircuitsGadgets](#)

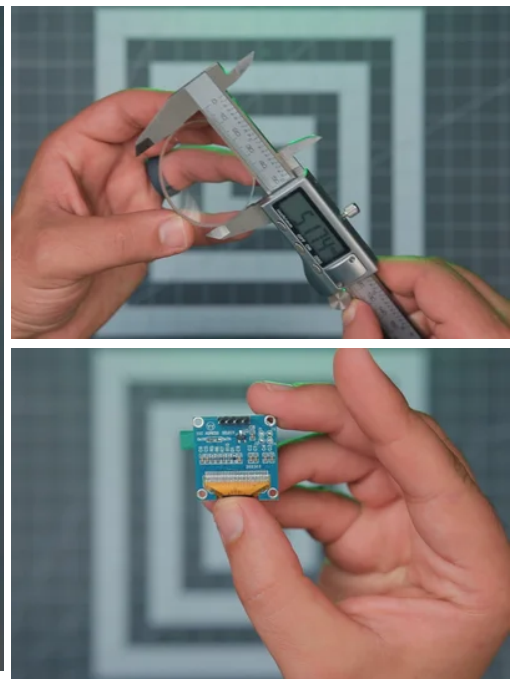
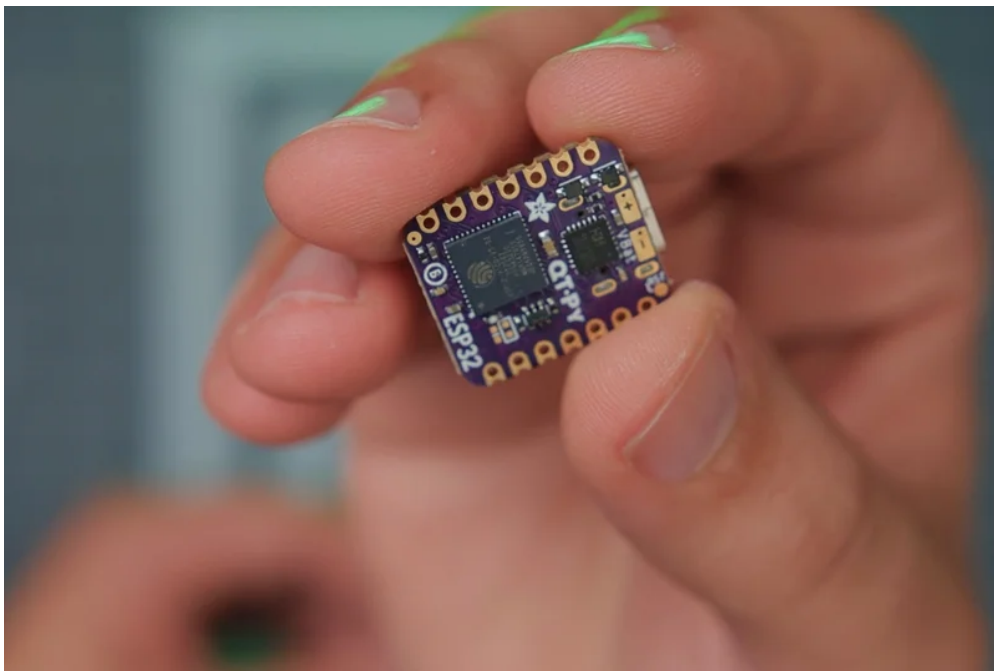


Introduction: Focus Dial — a Digital Distraction Blocker



Tired of constant distractions, I built the [Focus Dial](#)—my take on the failed [TimeChi crowdfunding project](#). It helps me work in time blocks by activating a Do Not Disturb mode on my phone and integrates with Home Assistant via webhook triggers to set lighting scenes and block distracting apps at the network level during focus sessions.

Supplies



Bill of materials

1. Adafruit QT Py ESP32 Pico [Adafruit](#)
2. NeoPixel Ring - 16 x 5050 [Adafruit](#)
3. OLED Module 128x64 White SSD1306 [Amazon](#)
4. Low Profile 56mm Encoder [Digikey](#)
5. 12mm Tactile Push Button [Amazon](#)
6. Hook-up wire [Amazon](#)
7. Heat shrink tubing [Amazon](#)
8. 50 mm Diameter Beaker Cover [Amazon](#)
9. M2 Self-Tapping Screws Assortment [Amazon](#)
10. Roller Bearings with Washers 30mm Bore 47mm [Amazon](#)
11. Rubber Feet [Amazon](#)
12. Tint Film (Black) [Amazon](#)
13. Rust-oleum Matte Clear [Amazon](#)
14. Silicone Mold Making Rubber [Amazon](#)
15. Titanium Dioxide Pigment [Amazon](#)

Tools

1. FDM 3D Printer [Bambu Lab A1](#)
2. Resin Printing System [Heygears](#)
3. Soldering Station/Iron. [Amazon](#)
4. Helping hands [Amazon](#)
5. Precision Screwdriver Set [Amazon](#)
6. Wire stripper [Amazon](#)
7. Sanding Sponges [Amazon](#)
8. Flux, Solder

9. Sandpaper Assortment
10. Flush cutters
11. Tweezers, pliers

Affiliate links may be included in this list. I may receive a small commission at no additional cost to you.

Step 1: Watch the Build Video

In the [project video](#), I walk you through the entire process of designing and building this. Watching it will provide an understanding of each step and how all the components fit together.

Step 2: Fabrication

You can download the [STL files](#) from the [GitHub repository](#) or choose your preferred format from the [Fusion embed](#).

Begin by 3D printing the enclosure parts. For the final version, I used a resin printer (Heygears Reflex), but these parts can also be printed on a [high-quality FDM 3D printer](#) with some adjustments. You might need to tweak the thin wall surrounding the encoder and bottom piece for better results.

If you decide not to make the diffuser out of silicone rubber like in the [video](#), now is a good time to 3D print it.

[Focus Dial](#)

Step 3: Post-processing

After printing the parts, I smoothed the visible surfaces using a 1000-grit foam [sanding strip](#). The resin print is already about 80% of the finish, so the sanding was minimal. If you opt for the FDM route and want a similarly smooth finish, check out [this project](#) that covers finishing techniques for FDM prints.

Once sanding was complete, I washed the parts with soap and water and let them to air dry in preparation for the clear coat. I used a matte clear spray (Rust-Oleum) and applied it in thin, even coats. Be sure to follow the drying and curing instructions provided on the can for the best results.

Step 4: Diffuser

As mentioned in [the video](#), I 3D-printed a mold to create a diffuser using [two-part silicone](#) and a small amount of [titanium dioxide](#). You only need a tiny amount in the mix to achieve a diffused look.

I let the silicone cure for 6 hours before carefully demolding it from the 3D-printed.

Step 5: Glass Lens

Next, I applied the [tint film](#) to the inside on the lens, trying to avoid creating any bubbles. I left the film oversized until most of it was applied, then used a hobby knife to carefully trace around and remove the excess.

For stubborn edges, applying a bit of heat can help secure them. Once the film was in place, I carefully removed the second transparent protective layer and set it aside for later.

Step 6: Assembly _1

To begin the assembly, I first got rid of the pins from the OLED module and trimmed the four corners to about 45° with flush cutters to ensure it fit into the [face piece](#). Next, I soldered four thin-gauge wires to the pads on the module and positioned it in place.

I then added a [backing piece](#) to the OLED and used black electrical tape to secure it. The tape also helps reduce light bleed from the back of the screen.

Next, I installed the LED ring into the [face piece](#) (as shown in the picture) and soldered slightly thicker wires for the power. The wire bundle can then be threaded through the [central shaft](#). Finally, I secured the [shaft](#) to the [face piece](#) using six (M2 x 4-5mm) self-tapping screws.

Step 7: Assembly _2

To prepare the encoder for the enclosure, I removed the ground leads and bent the three pins closer to the encoder body. Then, I soldered three wires to the pins and insulated them with heat shrink tubing.

Next, I threaded the three wires through the opening indicated by the arrow in the picture and snapped the encoder [into place](#). After that, I added the [central ring](#) and [bearings](#), securing the entire assembly to the previously assembled face piece using five (M2 x 10mm) self-tapping screws.

Be careful not to overtighten this part, as it could block the bearings from moving. Adjust the screws until you achieve a secure assembly with smooth rotation of the [central ring](#).

Step 8: Assembly _3

I used two small pieces of double-sided tape to secure the ESP32 in place for soldering. On the right side, I soldered the power wires, while on the left side, I connected the data wires for the LED ring, encoder, and button. For detailed connections, please refer to the wiring diagram.

One important note: I didn't leave a comfortable amount of space for the wires. The cables need to route over the three circles, staying as neat and flat as possible.

Next, I soldered two wires to the button, positioned it on the [small platform](#), and used it to divide the cables, keeping the wiring tidy.

The [button cap](#) serves two purposes: it prevents the button from falling out and covers the wires to avoid accidental snags. After placing the [button cap](#), I secured it to the assembly using three (M2 x 4-5mm) self-tapping screws.

Step 9: Assembly _4

To finish the assembly, I added three compression springs (approximately 6 x 10 mm) on the 3 screws in the [button cap](#). Next, I placed the [bottom button](#) and [bottom piece](#), securing them with four (M2 x 8mm) self-tapping screws. Ensure the orientation is correct by referring to the images.

The final step is to position the diffuser in its designated channel and place the lens on top. Once aligned, secure the [bezel](#) tightly to complete the assembly.

Step 10: Firmware

Set Up the IDE

1. **VS Code:**
2. If you don't already have it, download and install [Visual Studio Code](#).
3. **PlatformIO:**
4. Open VS Code.
5. Go to the Extensions Marketplace (Ctrl+Shift+X or Cmd+Shift+X).
6. Search for "PlatformIO" and install it if it's not already installed.
7. **Python:**
8. Ensure Python 3.6+ is installed on your system. If not, download it from [python.org](#) and add it to your system's PATH.
9. **Install Drivers (if needed):**
10. On **Windows**, you may need USB drivers for the ESP32 Pico QT Py to be recognized.
11. Download and install the drivers from [Adafruit's Drivers Page](#).

Build and Upload the Firmware

1. **Clone or Download the Repository:**
2. Clone the [repo from GitHub](#)
3. Or download the project as a ZIP and extract it.
4. **Open the Project in VS Code:**
5. In VS Code, go to *File > Open Folder...* and select the project folder.
6. **Connect the Focus Dial** to your computer with a **data-capable USB cable**.
7. **Select the Serial Port:**
8. Open the PlatformIO toolbar in VS Code.
9. Choose the serial port for your board from the drop-down menu.
10. **Build and Upload:**
11. Click **Build** to compile the firmware.
12. Click **Upload** to flash the firmware to the board.

After the build and upload, you should see **success messages** confirming the firmware was built and flashed. The Focus dial will now display a splash screen with a **spinner LED animation**. If you see this, everything is working correctly!

If you need to adjust pins assignments or other variables you can change them in [firmware/include/Config.h](#) and build/upload again.

Step 11: Usage

After the splash screen, the device will boot for the first time and greet you with the provisioning screen. In this mode, the ESP runs as an access point (AP), serving a webpage where you can configure the Wi-Fi credentials, set the webhook URL for your home automation trigger, and pair the device with your phone via Bluetooth for the janky personal automation trigger for IOS.

Note: Setting up Home Assistant and automation triggers is beyond the scope of this guide, but you can find some documentation [linked here](#).

Once provisioning is complete, the device transitions to the main screen, indicated by a blue breathing animation. To start a session, click the central button once or use the encoder to adjust the session duration.

When the timer starts, the device sends a "start" payload to the webhook task. To pause the timer, click the button once while it's running. To cancel the timer, click the button twice. When the timer runs out, the device sends a "stop" payload to complete the session.

To reset the device and re-trigger the provisioning process, hold down the main button for a few seconds while in the idle state.

Step 12: Done

That's it for this one you're done! Thank you for sticking around until the end.

I have other ideas and I plan on making projects like these [Follow me](#) for more.



[Youtube /@salimbenbouz](#)